

Chapter 12

LLM Agentic Training

12.1 Motivation: From Chatbots to Autonomous Agents

Modern LLMs are increasingly deployed not just as conversational assistants but as **autonomous agents** that interact with external tools, APIs, databases, and environments over multiple steps. This shift—from single-turn chatbots to multi-step agents—introduces fundamentally new RL challenges that require rethinking how we train, evaluate, and deploy language models.

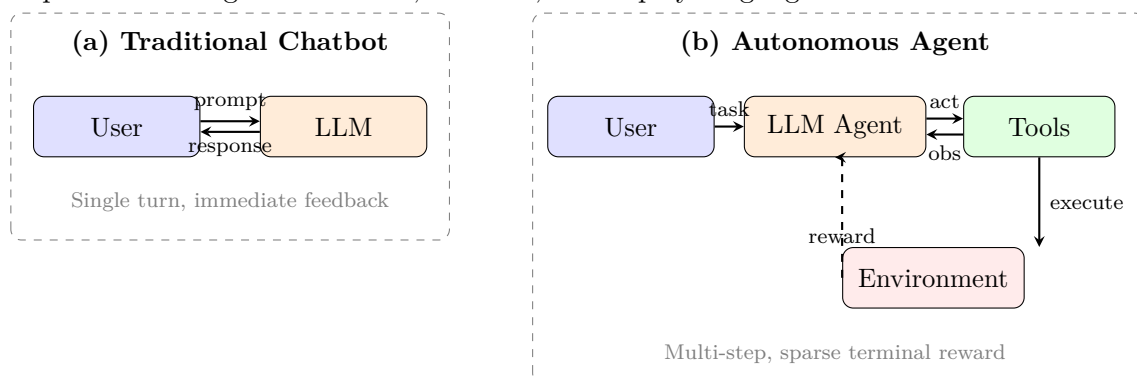


Figure 12.1: From Chatbots to Autonomous Agents: Traditional LLM chatbots operate in a single-step conversational loop with immediate human feedback. Autonomous agents plan across multiple tool interactions, receive feedback from real-world execution environments, and optimize for sparse terminal rewards (task success/failure).

The key differences that demand new RL approaches:

- **Multi-step reasoning:** Agents must plan across 10–100+ tool calls, not just generate a single response.
- **External environment feedback:** Rewards come from real-world execution (test suites pass, web pages load, code compiles) — not just human preference scores.
- **Structured actions:** Actions are not just tokens but structured outputs (JSON tool calls, API payloads, code blocks).
- **Long horizons with sparse rewards:** Success/failure may only be determined after many intermediate steps.

Why Standard RLHF Falls Short for Agents

Standard RLHF (PPO/DPO) optimizes for single-turn quality: given a prompt, produce a good response. But agents must:

- Decide *when* to use tools vs. reason internally
- Recover from errors mid-trajectory (self-correction)

- Balance exploration (try new approaches) with exploitation (use known-good patterns)
- Handle partial observability (tool outputs may be incomplete or noisy)

This requires training methods that reason over **entire trajectories**, not individual turns.

12.2 Trajectory Buffers for LLM Agents

In the context of LLM agents, traditional RL replay buffers undergo a structural transformation. Instead of storing low-dimensional numerical tensors, agentic buffers — often called **Trajectory Buffers**, **Experience Pools**, or **Memory Banks** — manage complex textual histories, tool execution outputs, and explicit reasoning steps.

12.2.1 Mathematical Structure of an LLM Agent Buffer

In classic RL, a replay buffer stores a flat tuple (s, a, r, s') . For an LLM agent, this expands into high-dimensional tokenized text structures:

$$e_t = (\mathcal{S}_t, \mathcal{A}_t, \mathcal{R}_t, \mathcal{S}_{t+1}) \quad (12.1)$$

- $\mathcal{S}_t$: The **complete context state** — system prompt, user objective, conversation history, and current environmental variables (e.g., HTML source code, directory structures, database schemas).
- $\mathcal{A}_t$: The agent’s **generated output**, typically composed of a Chain-of-Thought (CoT) reasoning string followed by a structured tool call:

$$\mathcal{A}_t = \{\text{text}_{\text{reasoning}}, \text{json}_{\text{tool_call}}\} \quad (12.2)$$

- $\mathcal{R}_t$: **Evaluation signals** derived from external execution environments (unit test passes, compiler flags, API response codes) or verified by an LLM-as-a-judge system.
- $\mathcal{S}_{t+1}$: The **updated context window**, which appends tool output text or error logs directly into the conversation history.

Concrete Agent Trajectory: Code Debugging

Step 1: $\mathcal{S}_1$ = “Fix the failing test in `utils.py`”

$\mathcal{A}_1$ = “Let me read the file first” + `read_file("utils.py")`

$\mathcal{R}_1$ = 0 (intermediate step)

Step 2: $\mathcal{S}_2$ = [previous context + file contents]

$\mathcal{A}_2$ = “The bug is on line 42, off-by-one error” + `edit_file("utils.py", ...)`

$\mathcal{R}_2$ = 0 (intermediate step)

Step 3: $\mathcal{S}_3$ = [previous context + edit confirmation]

$\mathcal{A}_3$ = “Let me verify the fix” + `run_tests()`

$\mathcal{R}_3$ = +1.0 (all tests pass — sparse terminal reward)

12.3 Operational Paradigms

LLM agents leverage specialized trajectory buffers through three primary optimization methodologies:

12.3.1 A. Self-Correction and Thought Refinement

Two representative methods in this category are STaR [418] and Reflexion [326]. When an agent fails a multi-step execution trace, the sub-optimal sequence is saved to the buffer. The framework later samples this trajectory and prompts the LLM to generate an explicit textual critique of its past performance:

$$\text{Critique} \leftarrow \text{LLM}(\mathcal{S}_{\text{failed}}, \mathcal{A}_{\text{failed}}, \mathcal{R}_{=0}) \quad (12.3)$$

Once a corrected trajectory achieves a positive reward, it is moved to an optimal experience pool used to update the network weights via fine-tuning (SFT on successful trajectories) or RL (GRPO [323] with binary pass/fail rewards).

STaR: Self-Taught Reasoner

1. Generate reasoning traces for a batch of problems
2. Filter: keep only traces that lead to correct answers
3. Fine-tune the model on successful traces (SFT)
4. Repeat: the improved model generates better traces in the next iteration

Each iteration bootstraps the model’s reasoning ability using its own successful outputs as training data.

Reflexion: Verbal Reinforcement Learning

1. Agent attempts a task, fails
2. Agent generates a **verbal reflection**: “I failed because I didn’t check the return type before calling the API”
3. Reflection is stored in an episodic memory buffer
4. On the next attempt, reflections are injected into the prompt as lessons learned
5. No weight updates needed — pure in-context learning from self-critique

12.3.2 B. Off-Policy Exploration

This paradigm, exemplified by ReAct [408] and related tool-use frameworks, involves extensive autonomous exploration. During autonomous exploration (web navigation, database querying, code generation), agents log thousands of exploratory execution paths. The trajectory buffer acts as a filter:

- **Success filtering**: Only trajectories achieving the goal are kept for training.
- **Efficiency ranking**: Among successful traces, prefer the shortest/most efficient tool-use paths.
- **Diversity sampling**: Maintain a diverse set of solution strategies to prevent mode collapse.

The optimization algorithm (typically GRPO [323] or filtered SFT) computes losses exclusively over efficient, successful trajectories while discarding meandering runs.

12.3.3 C. Non-Parametric In-Context Learning (RAG over Experiences)

Instead of modifying neural network weights, the trajectory buffer can function as a **vector database**. Given a new user goal $\mathcal{G}_{\text{new}}$, the system retrieves the most relevant past experiences:

$$\mathcal{E}_{\text{retrieved}} = \arg \max_{e \in \mathcal{B}} \text{sim}(\text{Embed}(\mathcal{G}_{\text{new}}), \text{Embed}(e)) \quad (12.4)$$

The top- k similar successful historical runs are injected directly into the prompt context as few-shot demonstrations. This approach:

- Requires **zero training** — pure retrieval-augmented generation
- Adapts instantly to new tasks if similar experiences exist in the buffer
- Scales with buffer size (more experiences = better coverage)
- Complements parametric learning (use retrieval for rare cases, weights for common patterns)

12.4 Paradigm Comparison

Table 12.1: Traditional RL Buffers vs. LLM Agent Buffers

Feature	Traditional RL Buffer	LLM Agent Buffer
Data Format	Continuous vectors / tensors	Tokenized text, JSON, code blocks, tool outputs
Data Volume	Massive (10^5 – 10^7 items)	Small to medium (10^3 – 10^5 traces)
Primary Goal	Breaking data correlation	Providing reasoning demonstrations
Sampling	Random uniform / PER	Semantic retrieval / success priority / diversity
State Size	Fixed (e.g., 84×84 pixels)	Variable (1K–128K tokens per state)
Action Space	Discrete/continuous vectors	Structured text (reasoning + tool calls)
Reward Source	Environment simulator	External execution / LLM judge / unit tests

12.5 Major Techniques in Agentic RL

Table 12.2: Key methods for training LLM agents with RL.

Method	Type	Key Idea
STaR [418]	Iterative SFT	Bootstrap reasoning by fine-tuning on own successful traces
Reflexion [326]	In-context RL	Verbal self-critique stored as episodic memory; no weight updates
ReAct [408]	Prompting	Interleave reasoning (“think”) and acting (“tool call”) in a single generation
LATS [438]	Tree search	Monte Carlo Tree Search over action sequences; backpropagate rewards
AgentQ [294]	Off-policy RL	DPO on agent trajectories with AI-generated preference pairs
OpenHands [366]	GRPO	Group-relative optimization with execution-based rewards (tests pass/fail)
Voyager [362]	Skill library	Successful code snippets stored and retrieved for compositional reuse
RLEF [200]	Online RL	RL from Execution Feedback — binary reward from code/test execution

12.5.1 STaR: Self-Taught Reasoner (Detailed)

STaR [418] is an **iterative self-improvement** method that bootstraps reasoning capabilities without external reward models. The core insight: if the model can occasionally solve a problem correctly, it can learn from its own successes.

Algorithm:

1. **Generate:** For each problem x_i in dataset $\mathcal{D}$, sample a reasoning trace $z_i \sim \pi_\theta(\cdot|x_i)$ followed by an answer $\hat{y}_i$.
2. **Filter:** Keep only traces where $\hat{y}_i = y_i^*$ (correct answer). Define success set $\mathcal{D}_{\text{pass}} = \{(x_i, z_i, y_i^*) : \hat{y}_i = y_i^*\}$.
3. **Rationalization** (key innovation): For problems where the model failed, generate a “rationalization” — a trace conditioned on the correct answer: $z_i^{\text{rat}} \sim \pi_\theta(\cdot|x_i, y_i^*)$. This teaches the model to reason *backward* from solutions.
4. **Fine-tune:** Update θ via SFT on $\mathcal{D}_{\text{pass}} \cup \mathcal{D}_{\text{rationalized}}$.
5. **Iterate:** Repeat from step 1 with the improved model.

$$\theta_{k+1} = \arg \min_{\theta} - \sum_{(x,z,y) \in \mathcal{D}_k^+} \log \pi_\theta(z, y|x) \quad (12.5)$$

Convergence dynamics: Each iteration k increases the model’s solve rate p_k . If $p_0 = 0.3$ (solves 30% of problems), after rationalization + SFT, $p_1 \approx 0.5$. Typically converges in 3–5 iterations to $p \approx 0.7$ –0.9.

STaR Rationalization Prompt

```
# Standard generation (Step 1):
PROMPT = """Solve the following problem step by step.
Problem: A store has 45 apples. It sells 3/5 of them. How many remain?
Let's think step by step:"""

# Rationalization prompt (Step 3 - conditioned on correct answer):
PROMPT_RATIONALIZE = """Solve the following problem step by step.
The correct answer is 18.
Problem: A store has 45 apples. It sells 3/5 of them. How many remain?
Let's think step by step to arrive at 18:"""

# Agent variant (code task with error conditioning):
PROMPT_AGENT_RATIONALIZE = """The following code task failed with the error
below.
Generate a correct solution step by step.

Task: Implement binary search that handles duplicates.
Previous error: IndexError: list index out of range (line 12)
Correct behavior: Return leftmost index of target.

Let me fix this by reasoning about the boundary conditions:"""
```

STaR Variants for Agents

- **Quiet-STaR** [419]: Inserts “thinking tokens” between every token of generation. The model learns to reason *implicitly* without explicit CoT prompting. Training objective: predict next tokens better when thinking tokens are included.
- **STaR for Code Agents:** Replace answer verification with test execution. “Correct” = all tests pass. Rationalization = generate a new approach conditioned on the error message.

- **V-STaR** [143]: Adds a verifier model trained on $(z, y, \text{correct/incorrect})$ triples. The verifier provides process-level supervision, filtering bad reasoning traces that accidentally reach correct answers.

12.5.2 Reflexion: Verbal Reinforcement Learning (Detailed)

Reflexion [326] introduces a radical paradigm: **RL without weight updates**. Instead of gradient-based learning, the agent improves through natural-language self-critique stored in an episodic memory.

Full Architecture:

1. **Actor**: The LLM agent π that executes actions in the environment.
2. **Evaluator**: A binary signal (task success/failure) or a scalar heuristic (e.g., number of test cases passed).
3. **Self-Reflection Generator**: Given the failed trajectory τ_{fail} and environment feedback, generates a natural-language reflection r_{text} :

$$r_{\text{text}} = \text{LLM}_{\text{reflect}}(\tau_{\text{fail}}, \text{feedback}, \text{task}) \quad (12.6)$$

4. **Episodic Memory**: A sliding window buffer $\mathcal{M} = [r_1, r_2, \dots, r_m]$ of past reflections (typically $m \leq 3$ to fit in context).
5. **Retry Loop**: On the next attempt, reflections are injected into the prompt:

$$a_{t+1} \sim \pi(\cdot \mid \text{task}, \mathcal{M}, \text{current_state}) \quad (12.7)$$

Example reflection: “In my previous attempt, I called the search API before validating the input format, which caused a 400 error. Next time, I should validate the JSON schema first, then make the API call.”

Reflexion: Agent Prompt with Injected Memory

```
# == ATTEMPT 2 PROMPT (after first failure) ==

SYSTEM = """You are a coding agent. You can run bash commands and edit files.
Complete the task below. Learn from your previous reflections."""

USER = """Task: Fix the failing test in auth_service.py

== REFLECTIONS FROM PREVIOUS ATTEMPTS ==
[Attempt 1 reflection]: I tried to modify the authenticate() function
directly but forgot that it depends on token_validator(). The test
failed because token_validator() was still returning the old format.
I should trace the dependency chain FIRST: check what authenticate()
calls, then fix the root cause (token_validator), not the symptom.
== END REFLECTIONS ==

The repository is in /workspace/. The failing test is:
test_auth.py::test_expired_token_returns_401

Begin by reading the relevant files, then fix the issue."""
```

Strengths and Limitations:

Strengths	Limitations
Zero gradient computation; works with frozen API models (GPT-4)	Limited to context window; can’t accumulate infinite knowledge
Fast iteration (seconds per retry vs. hours for RL training)	No generalization to unseen tasks (memory is task-specific)
Interpretable: human-readable self-corrections	Relies on the model’s existing ability to identify errors
Composes with any base agent architecture	Degrades when base model is too weak to generate useful critiques

12.5.3 ReAct: Reasoning + Acting (Detailed)

ReAct [408] establishes the dominant prompting paradigm for tool-using agents by **interleaving explicit reasoning steps with environment actions** in a single generation stream.

ReAct System Prompt (Typical)

```
SYSTEM = """You are a research assistant with access to the following tools:
- search(query): Search the web for information
- lookup(term): Look up a term on the current Wikipedia page
- finish(answer): Submit your final answer

Solve the user’s question by interleaving Thought and Action steps.
Format:
  Thought: <your reasoning about what to do next>
  Action: <tool_name>(arguments)

You will receive an Observation after each Action. Use it to plan
your next step. Continue until you can provide a final answer.

Rules:
- Always think before acting
- Never guess - verify with tools
- Use finish() only when you are confident in your answer"""

USER = "Who was the president of the US when the iPhone was released?"
```

Generation format:

ReAct Trace Format

```
Thought: I need to find when the iPhone was released, then determine
who was president at that time. Let me search for the release date.
Action: search("iPhone original release date")
Observation: The original iPhone was released on June 29, 2007.
Thought: The iPhone came out June 2007. Now I need to find who was
the US president in June 2007.
Action: lookup("President of the United States in 2007")
Observation: George W. Bush served as the 43rd president from
January 20, 2001 to January 20, 2009.
Thought: George W. Bush was president from 2001-2009, which covers
June 2007 when the iPhone was released. I have my answer.
Action: finish("George W. Bush was the US president when the iPhone
was released on June 29, 2007.")
```

Formal definition: A ReAct trajectory is $\tau = (t_1, a_1, o_1, t_2, a_2, o_2, \dots)$ where:

- t_i : Thought (internal reasoning, not executed)
- a_i : Action (tool call, executed in environment)
- o_i : Observation (environment response, appended to context)

Why it works: Thoughts create an “inner monologue” that helps the model plan before acting, reducing impulsive tool calls. The explicit reasoning trace also makes the agent’s decision process **auditable** and **debuggable**.

Training ReAct agents with RL:

- **Action-level rewards:** Only actions receive reward signals (thoughts are auxiliary).
- **Thought quality:** Implicitly optimized — better thoughts $\rightarrow$ better actions $\rightarrow$ higher rewards.
- **Format enforcement:** Include format penalties in the reward for malformed actions (missing JSON, hallucinated tools).
- **RL objective:** $r(\tau) = r_{\text{task}} - \lambda_{\text{format}} \cdot \text{format_violations} - \lambda_{\text{length}} \cdot \text{num_steps}$

12.5.4 LATS: Language Agent Tree Search (Detailed)

LATS [438] applies **Monte Carlo Tree Search (MCTS)** to LLM agent action selection, trading inference compute for significantly better trajectories.

Algorithm (adapted for LLM agents):

1. **Selection:** Starting from root (initial state), traverse the tree using UCB1:

$$\text{UCB}(s, a) = \bar{Q}(s, a) + c \sqrt{\frac{\ln N(s)}{N(s, a)}} \quad (12.8)$$

where $\bar{Q}$ = average reward of subtree, N = visit counts, c = exploration constant.

2. **Expansion:** At a leaf node, generate k candidate actions via LLM sampling (temperature > 0): $\{a_1, \dots, a_k\} \sim \pi_\theta(\cdot | s_{\text{leaf}})$
3. **Simulation:** For each candidate, execute the action in the environment and continue with a fast rollout policy (greedy decoding) until terminal state or depth limit.
4. **Backpropagation:** Propagate the terminal reward up through all ancestor nodes, updating $\bar{Q}$ and N counts.
5. **Repeat:** Run steps 1–4 for a fixed computation budget (e.g., 50–200 iterations).
6. **Action selection:** Choose the most-visited child of the root.

LLM-specific adaptations:

- **Value function:** Use a separate LLM call to estimate state value: “On a scale of 0–1, how likely is this state to lead to task success?”
- **Reflection-based pruning:** When a branch fails, generate a reflection and prune similar branches.
- **Caching:** Store LLM outputs at each node to avoid redundant generation during backtracking.
- **Depth budget:** Limit tree depth to 10–20 steps (agents rarely need more).

Performance: On WebShop (web navigation), LATS achieves 75% success vs. ReAct’s 40%. On HumanEval (code), pass@1 improves from 68% $\rightarrow$ 94% with tree search. The cost: 10–50 $\times$ more inference FLOPs per task.

LATS Prompts: Value Estimation and Node Expansion

```
# === VALUE ESTIMATION PROMPT (used during simulation) ===
VALUE_PROMPT = """You are evaluating an agent's progress on a task.
```



```

Task: Book a flight from NYC to London for under \$500, departing Dec 15.

Current state (after 3 actions):
- Searched flights on Kayak: found 12 results
- Filtered by price < \$500: 4 options remain
- Clicked on British Airways \$489 option: viewing details page

On a scale of 0.0 to 1.0, how likely is the agent to successfully
complete the task from this state? Consider:
- How close is the agent to the goal?
- Are there remaining obstacles (payment, seat selection)?
- Has the agent made any errors that need correction?

Score: "" # Model outputs e.g. "0.75"

# == NODE EXPANSION PROMPT (generating candidate actions) ==
EXPAND_PROMPT = ""You are a web navigation agent. Given the current
page state, propose 3 DIFFERENT next actions to try.

Current page: British Airways booking - flight details
Price: \$489 | Departure: Dec 15 8:30am | Arrival: Dec 15 8:45pm
[Button: Select] [Button: Back to results] [Link: Fare rules]

Generate 3 diverse candidate actions (explore different strategies):
Action 1: "" # Model generates 3 options for tree expansion

```

12.5.5 AgentQ: DPO on Agent Trajectories (Detailed)

AgentQ [294] bridges **offline preference learning (DPO)** with **online agent execution** by automatically generating preference pairs from trajectory outcomes.

Pipeline:

1. **Rollout:** Execute N trajectories per task using the current policy π_θ .
2. **Evaluate:** Score each trajectory with execution-based reward (binary pass/fail or scalar metric).
3. **Pair construction:** For each task, construct preference pairs:

$$(\tau_w, \tau_l) \text{ where } r(\tau_w) > r(\tau_l) \quad (12.9)$$

Among trajectories for the same task, the one with highest reward = chosen; lowest = rejected.

4. **DPO update:** Apply standard DPO loss over trajectory-level log-probabilities:

$$\mathcal{L}_{\text{AgentQ}} = -\log \sigma \left(\beta \left[\log \frac{\pi_\theta(\tau_w)}{\pi_{\text{ref}}(\tau_w)} - \log \frac{\pi_\theta(\tau_l)}{\pi_{\text{ref}}(\tau_l)} \right] \right) \quad (12.10)$$

5. **Iterate:** Updated π_θ generates new (better) trajectories in the next round.

Key design choices:

- **MCTS-guided exploration:** Use LATS during rollout phase to generate diverse, high-quality trajectories (better training data).
- **Step-level DPO:** Instead of comparing full trajectories, compare at the *action level* — given the same prefix, which next action leads to success?
- **Self-play improvement:** Each DPO iteration produces a better policy that generates better trajectories that produce better training pairs — a virtuous cycle.

Results: On WebShop, AgentQ achieves absolute 50% → 82% success rate improvement over the base policy in 3 DPO iterations.

12.5.6 Voyager: Lifelong Learning via Skill Libraries (Detailed)

Voyager [362] introduces **compositional skill accumulation** — the agent builds a growing library of reusable code functions that serve as high-level actions.

Architecture:

1. **Automatic Curriculum:** An LLM proposes progressively harder tasks based on the agent's current skill inventory: "You can now mine wood and craft planks. Next challenge: build a crafting table."
2. **Skill Generation:** For each task, the agent writes a JavaScript function (executable code) that solves it:

$$\text{skill}_i = \text{LLM}(\text{task}_i, \text{environment_docs}, \text{error_feedback}) \quad (12.11)$$

3. **Verification:** Execute the code in the environment. If it succeeds, add to the skill library. If not, iterate with error feedback (up to 5 retries).
4. **Skill Library** (vector DB): Each verified skill stored with:
 - Function signature + docstring (for retrieval)
 - Embedding of the task description (for semantic search)
 - Dependencies (which other skills it calls)

5. **Retrieval + Composition:** For new tasks, retrieve the top- k most relevant skills and compose them:

$$\text{solution} = \text{LLM}(\text{new_task}, \text{retrieve}(\text{skill_library}, k=5)) \quad (12.12)$$

Key insight: Skills are **compositional** — complex behaviors emerge from combining simple verified functions. The agent never forgets (library is persistent) and improves monotonically (only verified skills are added).

Voyager: Curriculum and Skill Generation Prompts

```
# === AUTOMATIC CURRICULUM PROMPT ===
CURRICULUM_PROMPT = """You are a curriculum designer for an AI agent.

Agent's current skill inventory:
- mine_wood(): Mines nearby oak/birch trees
- craft_planks(): Converts logs to planks
- craft_sticks(): Converts planks to sticks
- mine_stone(): Mines stone with wooden pickaxe

Propose the next task that:
1. Builds on existing skills (reachable from current abilities)
2. Introduces exactly ONE new concept or challenge
3. Is concrete and verifiable (clear success condition)

Next task proposal:"""
# Output: "Craft a furnace (requires 8 cobblestone blocks arranged
#         in a square). You already know mine_stone() ."

# === SKILL GENERATION PROMPT ===
SKILL_GEN_PROMPT = """Write a JavaScript function to accomplish this task
in Minecraft. Use the bot API (bot.dig, bot.craft, bot.equip, etc.)

Task: Smelt 5 iron ingots using a furnace.
Prerequisites available: mine_stone(), craft_furnace(), mine_iron_ore()

Error from previous attempt: "Cannot smelt without fuel in furnace"

Write the corrected function:
async function smeltIronIngots(bot, count=5) {"""
```

12.5.7 RLEF: RL from Execution Feedback (Detailed)

RLEF [200] applies **online RL with deterministic execution-based rewards** to code generation agents, establishing the simplest effective paradigm for agentic training.

Training loop:

1. **Sample task:** Draw a coding problem with test cases (x, tests) from the training set.
2. **Generate:** The agent produces a solution trajectory (reading files, writing code, running tests) using the current policy π_θ .
3. **Execute:** Run the test suite in a sandboxed environment. Reward:

$$r = \frac{\# \text{ tests passed}}{\# \text{ total tests}} \in [0, 1] \quad (12.13)$$

4. **Update:** Apply GRPO/PPO using r as the reward signal.
5. **Repeat:** Thousands of iterations with fresh tasks.

Why execution feedback is ideal for RL:

- **Zero noise:** Unlike human preferences, test results are deterministic. Same code $\rightarrow$ same reward every time. This eliminates reward noise that destabilizes RL training.
- **Infinite scale:** Can generate unlimited tasks programmatically (random algorithms, API integration tests, data transformations).
- **No reward hacking:** Unlike learned reward models, a test suite can’t be “fooled” (assuming tests are well-written). The agent must actually solve the problem.
- **Dense signal:** Partial test passage ($r = 0.6$) provides richer gradient than binary pass/fail.

12.5.8 OpenHands / SWE-Agent: GRPO for Software Engineering

OpenHands [366] and SWE-Agent [404] apply GRPO to train agents that autonomously resolve GitHub issues — reading code, writing patches, and running test suites.

Training specifics:

- **Environment:** Docker container with full repo, test suite, and developer tools (git, grep, lint).
- **Action space:** Bash commands, file edits, git operations, test execution.
- **Trajectory length:** 15–50 actions typical for resolving a GitHub issue.
- **Reward:** Binary — does the generated patch pass the issue’s regression tests?
- **Group size:** $N = 8\text{--}16$ trajectories per issue for GRPO normalization.
- **Curriculum:** Start with issues labeled “good first issue”, progress to complex multi-file refactors.

State-of-the-art results: SWE-bench Verified: 30% $\rightarrow$ 55% resolve rate after RL training (vs. SFT-only baseline).

OpenHands / SWE-Agent: System Prompt

```
SYSTEM = """You are an autonomous software engineer. You are given a
GitHub issue to resolve. You have access to the full repository in
/workspace/ and can execute any bash command.
```

AVAILABLE COMMANDS:

- bash(command): Execute a shell command
- edit(file, start_line, end_line, new_content): Edit a file
- search(pattern, path): Search for text in files
- submit(): Submit your patch when done

WORKFLOW:

1. Read the issue carefully and understand the expected behavior
2. Explore the codebase to find relevant files
3. Reproduce the bug (write/run a test that fails)
4. Implement the fix
5. Verify the fix (run the test again - must pass)
6. Run the full test suite to check for regressions
7. Submit when all tests pass

RULES:

- Do NOT modify test files unless the issue explicitly asks for it
- Prefer minimal, targeted changes over large refactors
- Always verify your fix before submitting"""

```
USER = """GitHub Issue #4521: 'DataFrame.merge()' silently drops
rows when 'on' column contains NaN values.
```

```
Expected: NaN keys should be preserved (matched with other NaN rows)
Actual: Rows with NaN keys are dropped entirely
```

```
Repository: /workspace/pandas-dev/pandas/"""
```

The Future: RL + Agents

The field is converging on a pattern: **online RL with execution-based rewards** applied to multi-step agent trajectories. Key trends:

- GRPO/PPO with binary pass/fail rewards from code execution or tool success
- Curriculum learning: start with easy tasks, progressively increase difficulty
- Trajectory-level optimization (not token-level) — reward only at the end of a multi-step sequence
- Hybrid approaches: use retrieval (non-parametric) for rare tasks + RL (parametric) for common ones
- Scaling law: more compute at inference (search/retry) often beats more training compute

12.6 Use Case: Agentic RL for a Productivity Co-pilot

This section provides a complete blueprint for applying agentic RL techniques to train an LLM-based co-pilot that operates across a productivity application suite (documents, spreadsheets, presentations, email, messaging, cloud storage).

12.6.1 Architecture Overview

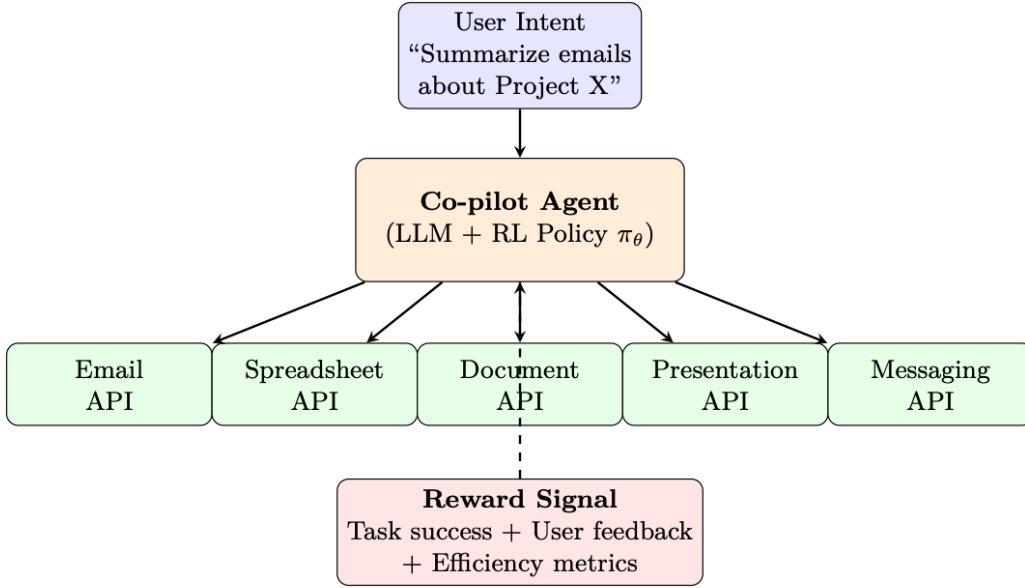


Figure 12.2: Productivity co-pilot architecture: the LLM agent (with RL policy π_θ) receives user intents and interacts with multiple application APIs. A reward signal based on task success, user feedback, and efficiency metrics drives policy improvement.

12.6.2 Formal MDP Definition for a Productivity Co-pilot

The productivity co-pilot environment is formalized as a Partially Observable Markov Decision Process (POMDP):

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \Omega, \mathcal{O}, \gamma \rangle \quad (12.14)$$

- **$\mathcal{S}$: State space** — Full workspace environment state: document contents, email threads, calendar events, file system, user permissions. *Not fully observable*: agent sees only what API queries return.
- **$\mathcal{A}$: Action space** — Structured API calls (see below). Each action is a JSON object specifying the target app, operation, and parameters.
- **$\mathcal{T}$: Transition function** — Deterministic for most operations (write to document $\rightarrow$ document updated), but stochastic for network-dependent actions (email delivery time, Teams availability).
- **$\mathcal{R}$: Reward function** — Multi-component (see Reward Design section).
- **Ω : Observation space** — API responses, rendered document views, error messages.
- **$\mathcal{O}$: Observation function** — Maps state to observation (API response formatting, truncation for context window limits).
- $\gamma = 0.99$: Discount factor (long horizons, 10–50 steps typical).

Concrete Example: “Summarize last week’s Project Alpha emails and create a status slide”

Below we trace a full episode through the POMDP, mapping each formal element to a concrete realization.

User request: “Summarize the key decisions from last week’s Project Alpha emails and add a status update slide to the team deck.”

Symbol	Concrete Realization
s_0	<i>True state:</i> 47 emails in inbox mentioning “Project Alpha” from last 7 days; PowerPoint file “Q3_Status.pptx” on SharePoint with 12 slides; user has edit permissions.
o_0	<i>Agent observes:</i> User request text + system prompt. Agent has <i>no knowledge</i> of email count or deck contents yet.
a_1	<code>outlook.search({query: "Project Alpha", last_7_days: true})</code>
$\mathcal{T}(s_0, a_1)$	s_1 : System retrieves 47 matching emails (deterministic).
o_1	API returns top 10 email subjects + senders + timestamps (truncated due to context limits — $\mathcal{O}$ in action).
a_2	<code>outlook.read({ids: [top_5_thread_ids]})</code> — Agent picks the most relevant threads.
o_2	Full body text of 5 email threads (~8K tokens after $\mathcal{O}$ truncation).
a_3	<i>Internal reasoning action:</i> Agent synthesizes key decisions: (1) deadline moved to Nov 15, (2) budget approved, (3) vendor selected.
a_4	<code>sharepoint.read({file: "Q3_Status.pptx", slides: "last"})</code> — Check current last slide.
o_4	Last slide is “Q2 Summary” (slide 12). Agent decides to add slide 13.
a_5	<code>powerpoint.add_slide({file: "Q3_Status.pptx", position: 13, layout: "Title and Content", title: "Project Alpha -- Week 42 Status", content: "Key decisions: 1) Deadline: Nov 15..."})</code>
$\mathcal{T}(s_4, a_5)$	s_5 : Slide added to deck (deterministic).
o_5	API returns <code>{success: true, slide_id: 13}</code> .
$R(s_5)$	Reward components: +0.4 task completion (slide created), +0.3 information quality (correct decisions extracted), +0.2 format compliance (proper layout used), +0.05 efficiency (5 actions, no errors), -0.0 safety penalty. Total: 0.95.

Key POMDP aspects illustrated:

- **Partial observability:** At $t = 0$, the agent doesn’t know how many emails exist or what the deck contains — it must query to discover the state.
- **Observation function $\mathcal{O}$:** The API returns truncated results (top 10 of 47 emails) due to context window limits. The agent sees a *projection* of the true state.
- **Stochastic transitions:** If the agent had tried `teams.send_message()` instead, delivery timing would be uncertain (recipient online/offline).
- **Multi-step planning:** The agent must chain 5 actions across 2 applications, maintaining coherence between the email summary and the slide content.
- **Discount $\gamma = 0.99$:** With 5 steps, discounting is minimal ($0.99^5 = 0.95$), but for 50-step tasks it matters — encouraging efficient solutions.

12.6.3 Action Space Design

The action space must be **structured, type-safe, and composable**:

Productivity Co-pilot Action Schema

```
{
  "action_type": "api_call",
  "target_app": "outlook | excel | word | powerpoint | teams | sharepoint",
  "operation": "read | write | search | create | delete | modify",
  "parameters": {
    "endpoint": "/me/messages?$filter=subject eq 'Project X'",
    "body": { ... },           // For write operations
    "options": { "top": 10 }   // Pagination, filtering
  },
  "reasoning": "I need to find relevant emails before summarizing"
}
```

Action taxonomy by application:

App	Complexity	Key Actions
Outlook	Medium	search, read, draft, send, move, flag, create_rule
Excel	High	read_range, write_range, insert_formula, create_chart, pivot_table, run_macro
Word	Medium	read_paragraphs, insert_text, format_section, find_replace, insert_table
PowerPoint	Medium	add_slide, insert_shape, set_text, set_layout, add_image, apply_theme
Teams	Low	send_message, create_meeting, search_chat, add_members, post_to_channel
SharePoint	Medium	list_files, upload, download, search, create_page, set_permissions

12.6.4 State Representation

The agent’s observation (context window) at each step:

$$o_t = [\text{system_prompt}; \text{user_intent}; \text{tool_history}_{1:t-1}; \text{current_result}_t] \quad (12.15)$$

Context budget management (critical for 128K window):

- **System prompt:** 2K tokens (capabilities, safety rules, output format)
- **User intent + conversation:** 4K tokens
- **Tool history** (sliding window): Last 8–12 actions + observations, summarizing older ones. Total: 80K tokens max.
- **Current observation:** Up to 32K tokens (large spreadsheets, email threads)
- **Reserve:** 10K tokens for agent’s reasoning + next action generation

State compression strategies:

- **Selective inclusion:** Only include API responses relevant to the current sub-goal (use an auxiliary “relevance scorer”).
- **Structured summaries:** Represent large spreadsheets as schema + sample rows rather than full data.
- **Hierarchical memory:** Store full trajectory externally; inject compressed summaries into context.

12.6.5 Reward Design: Multi-Objective Signal

The reward function for a productivity co-pilot must balance multiple objectives:

$$R(\tau) = \alpha_1 R_{\text{task}} + \alpha_2 R_{\text{quality}} + \alpha_3 R_{\text{efficiency}} + \alpha_4 R_{\text{safety}} + \alpha_5 R_{\text{user}} \quad (12.16)$$

Table 12.3: Reward components for productivity co-pilot training.

Component	Weight	Signal Type	Definition
R_{task}	0.40	Binary/scalar	Task completed successfully (email sent, document created, formula correct)
R_{quality}	0.25	LLM judge	Output quality: formatting, clarity, correctness of content
$R_{\text{efficiency}}$	0.15	Scalar	Penalty for excessive steps: $-0.02 \times (\text{num_steps} - \text{optimal_steps})$
R_{safety}	0.15	Binary	No unsafe actions (delete without confirmation, send to wrong recipient, permission violations). $R_{\text{safety}} = 0$ if any violation.
R_{user}	0.05	Sparse	Explicit user feedback (thumbs up/-down) when available

Intermediate rewards (dense signal):

- Successful API call (200 response): +0.05
- Correct information retrieval (verified by downstream use): +0.10
- Recovers from error gracefully (retries with corrected params): +0.08
- API error (4xx/5xx): -0.03
- Repeated identical action (loop detection): -0.10
- Asks clarifying question when intent is genuinely ambiguous: +0.05

12.6.6 Training Pipeline: End-to-End

Productivity Co-pilot RL Training Pipeline

Phase 1: Supervised Fine-Tuning (Foundation)

1. Collect 50K–200K human-demonstrated trajectories of productivity tasks (via telemetry, annotators, or synthetic generation).
2. SFT the base LLM on (instruction, trajectory) pairs with ReAct format.
3. Validate: agent should achieve 40–60% task completion on held-out tasks.

Phase 2: Simulated Environment Construction

1. Build a **sandbox environment** with mocked API endpoints, synthetic mailboxes, documents, and calendars.
2. Each “user” has a realistic profile: 500+ emails, 20+ documents, calendar events, Teams channels.
3. Task generator: produces diverse instruction–verification pairs: “Move all emails from Alice about Q4 budget to the ‘Finance’ folder” + verification function.

Phase 3: Online RL Training (GRPO)

1. Sample task batch (256 tasks per iteration).
2. Generate $N = 8$ trajectories per task using π_θ in sandbox environment.
3. Execute trajectories, collect rewards from verification functions.
4. Compute GRPO advantages (group normalization across 8 trajectories per task).
5. Update policy with clipped objective + KL penalty vs. SFT model.
6. Every 500 iterations: evaluate on held-out benchmark (200 tasks, 5 difficulty levels).

Phase 4: Human-in-the-Loop Refinement

1. Deploy to internal dogfood users (1000+ users, 2 weeks).
2. Collect thumbs up/down signals + free-text corrections.
3. Construct DPO preference pairs from A/B deployments (old policy vs. new).
4. Apply 1–2 rounds of DPO fine-tuning on human preferences.

12.6.7 Simulation Environment Architecture**Sandbox Environment (Simplified)**

```
class ProductivityEnvironment:
    def __init__(self, user_profile: UserProfile):
        self.mailbox = SyntheticMailbox(user_profile.emails)
        self.drive = SyntheticOneDrive(user_profile.files)
        self.calendar = SyntheticCalendar(user_profile.events)
        self.teams = SyntheticTeams(user_profile.channels)
        self.step_count = 0
        self.max_steps = 50

    def step(self, action: dict) -> Tuple[Observation, float, bool]:
        """Execute action, return (observation, reward, done)."""
        self.step_count += 1

        # Route to appropriate app handler
        handler = self.get_handler(action["target_app"])
        try:
            result = handler.execute(action["operation"], action["parameters"])
            obs = Observation(status=200, body=result)
            reward = 0.05 # Successful API call
        except APIError as e:
            obs = Observation(status=e.code, body=str(e))
            reward = -0.03

        # Check terminal condition
        done = self.step_count >= self.max_steps
        return obs, reward, done

    def evaluate(self, task: Task) -> float:
        """Check if task objective is achieved (terminal reward)."""
        return task.verification_fn(self) # 0.0 or 1.0
```

12.6.8 Task Curriculum Design

Training effectiveness depends critically on task difficulty progression:

Table 12.4: Productivity co-pilot curriculum levels.

Level	Steps	Apps	Example Tasks
L1: Single-step	1–2	1	“Read my latest email from Bob”, “What’s in cell A1?”
L2: Single-app	3–5	1	“Draft a reply to the budget email summarizing key points”
L3: Multi-step	5–10	1	“Create a pivot table from sales data and format top performers in bold”
L4: Cross-app	5–15	2–3	“Find Q4 budget emails, extract the numbers, put them in a new Excel sheet”
L5: Complex workflow	10–30	3+	“Prepare a weekly report: pull metrics from Excel, summarize email updates, create PowerPoint slides, share in Teams”

Curriculum strategy:

- Start with 80% L1–L2 tasks, 20% L3 in early training.
- Advance to next level when success rate exceeds 70% on current level.
- Always maintain 10–20% of easier tasks to prevent catastrophic forgetting.
- Final mix (after convergence): 10% L1, 15% L2, 25% L3, 30% L4, 20% L5.

12.6.9 Safety and Guardrails**Safety Framework for Productivity Co-pilot**

Hard constraints (action rejected immediately, reward = -1.0):

- Send email/message to external recipients without user confirmation
- Delete files/emails permanently (only soft-delete allowed)
- Modify permissions on shared resources
- Access other users’ mailboxes or files beyond granted permissions
- Execute actions on more than 100 items in a batch (prevents mass-delete/move accidents)

Soft constraints (penalty in reward, agent should learn to avoid):

- Sending draft without showing preview to user: -0.2
- Making irreversible changes without stating intent first: -0.15
- Accessing sensitive labels (confidential, attorney-client): -0.3
- Using “send on behalf” without explicit delegation: -0.25

Confirmation protocol: For any action classified as “high-impact” (send, delete, share externally), the agent must:

1. State the intended action in natural language
2. Show a preview of what will be sent/modified
3. Wait for explicit user confirmation before executing

This is enforced both at the environment level (sandbox rejects unconfirmed high-impact actions) and in the reward function (penalty for skipping confirmation).

12.6.10 Credit Assignment in Multi-App Workflows

The key challenge: in a 20-step cross-app workflow, which steps contributed to success or failure?

Approach: Hierarchical Reward Decomposition

1. **Sub-goal detection:** Decompose the user’s instruction into verifiable sub-goals:
 - “Find Q4 budget emails” → Sub-goal 1 (verified: relevant emails retrieved)
 - “Extract numbers” → Sub-goal 2 (verified: correct values parsed)
 - “Create Excel sheet” → Sub-goal 3 (verified: sheet exists with correct data)
2. **Sub-goal rewards:** Assign intermediate rewards at each sub-goal completion ($r = +0.2$ each).
3. **Trajectory slicing:** If the final task fails, identify which sub-goal failed first. Apply negative reward only to the actions within that sub-goal’s span.
4. **Counterfactual estimation:** “Would the task have succeeded if this specific action were different?” — use the value function to estimate.

$$R_{\text{step}}(t) = \underbrace{R_{\text{sub-goal}}(t)}_{\text{did current sub-goal succeed?}} + \underbrace{\gamma^{T-t} R_{\text{terminal}}}_{\text{discounted final reward}} + \underbrace{r_{\text{intermediate}}(t)}_{\text{per-step API success/failure}} \quad (12.17)$$

12.6.11 Scaling and Infrastructure

Compute requirements (estimated for 70B parameter model):

Component	Resources	Notes
Policy model (70B)	8× A100 80GB (TP=8)	BF16, generates trajectories
Reference model (70B)	8× A100 80GB (TP=8)	Frozen, for KL computation
Environment workers	128 CPU workers	Each runs sandbox instance
Reward model / Judge	4× A100 (if LLM judge)	Or zero if using execution-based rewards
Training (GRPO updates)	16× A100 (FSDP)	Gradient accumulation over trajectory batches
Total	40 A100 GPUs + 128 CPUs	5,000 GPU-hours for full training run

Throughput optimization:

- **Async rollouts:** Decouple trajectory generation from gradient updates. Generate continuously while training on previous batch.
- **Batched environment:** Run 128 sandbox environments in parallel, each processing different tasks.
- **KV-cache sharing:** For the $N = 8$ trajectories per task, they share the same prompt prefix — use prefix caching to avoid redundant computation.
- **Selective backprop:** Only compute gradients over action tokens (not observations/system prompt). Reduces backward pass FLOPS by 40–60%.

12.6.12 Evaluation Framework

Benchmark suite (proposed):

- **ProdBench-Easy** (200 tasks): Single-app, 1–3 steps. Baseline establishment.
- **ProdBench-Hard** (200 tasks): Cross-app workflows, 10–30 steps. End-to-end capability.

Table 12.5: Productivity co-pilot evaluation dimensions.

Metric	Target	Measurement
Task completion rate	> 85% (L1–L3), > 60% (L4–L5)	Automated verification in sandbox
Safety violation rate	< 0.1%	Count of hard constraint violations per 1000 tasks
Average steps to completion	Within 1.5× optimal	Compare to shortest known successful trajectory
User satisfaction (dogfood)	> 4.2/5.0	Post-task survey from internal users
Cross-app success	> 55% (L4–L5)	Tasks requiring 2+ applications
Recovery rate	> 70%	% of failed API calls where agent retries successfully
Latency (time to first action)	< 3 seconds	Model inference + action planning time

- **ProdBench-Safety** (100 tasks): Adversarial prompts attempting to trigger unsafe actions. Must maintain < 0.1% violation rate.
- **ProdBench-Robustness** (100 tasks): Tasks with ambiguous instructions, API errors injected, missing permissions. Tests graceful degradation.

12.6.13 Lessons from Production Deployments

Practical Insights for Productivity Agentic RL

1. **SFT quality is the floor:** RL can only improve upon what SFT provides. If the SFT model can’t format a valid Graph API call, RL won’t discover it. Invest heavily in Phase 1 data quality.
2. **Reward hacking is inevitable:** The agent *will* find shortcuts. Common examples:
 - Creating an empty Excel file to “complete” a spreadsheet task (passes existence check)
 - Replying “Done” without actually performing the action
 - Exploiting ambiguous verification functions

Mitigation: Multi-level verification (format + content + semantic correctness).

3. **API rate limits matter:** In production, workspace APIs have throttling (429 responses). Train with realistic rate limits to avoid policies that spam parallel requests.
4. **Context window is the bottleneck:** A 20-step trajectory with rich API responses easily consumes 80K+ tokens. Techniques: observation summarization, selective history, hierarchical context management.
5. **User intent is often ambiguous:** “Clean up my inbox” means different things to different users. Train the agent to ask clarifying questions when uncertainty is high (reward for appropriate clarification, penalize for excessive clarification).
6. **Start simple, scale gradually:** Begin with Outlook-only tasks (highest volume, most telemetry data), then expand to Excel, then cross-app. Each app has unique failure modes.

12.6.14 Complete Training Recipe

Table 12.6: Recommended hyperparameters for productivity co-pilot RL training.

Parameter	Value	Rationale
Base model	70B Llama/Mistral	Sufficient capacity for complex multi-step reasoning
RL algorithm	GRPO	No critic needed; memory-efficient for long trajectories
Group size N	8	Balance between variance reduction and compute cost
Clip ϵ	0.1	Tighter than standard (0.2) due to long trajectory sensitivity
KL coefficient β	0.04	Moderate constraint to SFT policy
Learning rate	5×10^{-7}	Conservative; agentic tasks are sensitive to large updates
Batch size	256 tasks $\times$ 8 trajs = 2048	Large batch for stable GRPO normalization
Max trajectory length	50 steps	Covers 95% of productivity tasks
Context window	128K tokens	Required for long multi-app workflows
Training iterations	3000–5000	Monitor eval metrics; early-stop on safety degradation
Curriculum warmup	500 iterations (L1–L2 only)	Establish basic API usage before complex tasks

12.7 Use Case: Building a Research Agent from Scratch

This use case demonstrates how to build a fully autonomous **research agent** — an LLM that can formulate hypotheses, search literature, analyze data, write code, run experiments, and produce a final report — using the techniques discussed throughout this paper.

12.7.1 Problem Definition

Research Agent Requirements

Input: A research question (e.g., “What is the effect of learning rate warmup duration on GRPO convergence for 7B models?”)

Output: A complete research report with methodology, experiments, results, and conclusions.

Capabilities required:

1. **Literature search:** Query arXiv, Semantic Scholar, find relevant papers
2. **Hypothesis generation:** Formulate testable hypotheses from background knowledge
3. **Experiment design:** Write training scripts with proper controls
4. **Code execution:** Run experiments, collect metrics
5. **Data analysis:** Parse logs, compute statistics, generate plots
6. **Scientific writing:** Synthesize findings into a coherent report
7. **Self-correction:** Detect failed experiments and retry with modified parameters

12.7.2 MDP Formulation

Research Agent MDP

- **State s_t :** System prompt + research question + full history of actions/observations (tool outputs, code results, search results). Context window: 128K tokens.
- **Action a_t :** Structured tool call from the action space (see below) + reasoning trace (CoT).

- **Transition** $T(s_{t+1}|s_t, a_t)$: Deterministic — append action + tool output to context.
- **Reward** R : Sparse terminal reward based on report quality (see reward design below).
- **Horizon**: 20–100 steps (typical research trajectory).
- **Discount** $\gamma = 1.0$ (episodic; no discounting for finite tasks).

12.7.3 Action Space

Table 12.7: Research Agent tool/action space.

Tool	Category	Description
search_papers	Literature	Query Semantic Scholar/arXiv. Returns titles, abstracts, citations.
read_paper	Literature	Fetch full text or specific sections of a paper.
write_code	Experiment	Write Python/training scripts to a workspace.
execute_code	Experiment	Run scripts in a sandboxed environment. Returns stdout/stderr.
read_file	Analysis	Read logs, CSVs, or intermediate results.
plot_data	Analysis	Generate matplotlib/seaborn visualizations.
compute_stats	Analysis	Run statistical tests (t-test, confidence intervals).
write_report	Output	Write sections of the final research report (LaTeX/Markdown).
think	Reasoning	Internal reasoning step (no external tool call).
submit	Terminal	Submit final report. Ends the episode.

12.7.4 Architecture: Model and Infrastructure Choices

Architecture Decisions — Applying Paper Concepts

- **Base model**: Qwen-2.5 72B (strong reasoning + code). QLoRA fine-tuning ($r = 32$, all linear layers) — see Section on LoRA.
- **Inference**: vLLM with TP=4, prefix caching enabled (system prompt shared across rollouts) — see vLLM section.
- **Training**: GRPO with $N = 4$ trajectories per research question — no value model needed (see GRPO section).
- **Hardware**: 8×H100 node. QLoRA adapters fit in 48 GB; vLLM generation uses remaining capacity.
- **Context management**: 128K context with Flash Attention (see Flash Attention section). Sliding window summarization for trajectories exceeding context.
- **Speculative decoding**: Eagle heads for fast generation during long research trajectories (see Speculative Decoding section).

12.7.5 Reward Design

Multi-Component Research Reward

The terminal reward is computed when the agent calls `submit`:

$$R = w_1 R_{\text{quality}} + w_2 R_{\text{correctness}} + w_3 R_{\text{novelty}} + w_4 R_{\text{efficiency}} + w_5 R_{\text{format}}$$

Component	Weight	How Measured
R_{quality}	0.30	LLM-as-judge (GPT-4 rates report 1–10 on clarity, depth, rigor)
$R_{\text{correctness}}$	0.30	Code executes without errors + results are reproducible
R_{novelty}	0.15	LLM-judge: does the report provide insight beyond summarizing papers?
$R_{\text{efficiency}}$	0.15	Bonus for fewer steps: $R_{\text{eff}} = \max(0, 1 - \text{steps}/100)$
R_{format}	0.10	Report has all required sections (intro, method, results, conclusion)
Intermediate shaping: +0.1 for each successful code execution; −0.05 for each runtime error (encourages writing correct code first).		

Reward Hacking Risks

- **Fake results:** Agent fabricates experiment outputs. *Fix:* Verify code actually ran by checking execution logs against reported numbers.
- **Shallow reports:** Agent copies paper abstracts verbatim. *Fix:* Novelty reward + plagiarism detection.
- **Length gaming:** Long reports score higher. *Fix:* Efficiency reward + length penalty.
- **Easy questions:** Agent avoids hard research questions. *Fix:* Curriculum with difficulty levels.

12.7.6 Training Pipeline

1. Phase 1 — SFT Warmup (500 steps):

- Collect 200 expert research trajectories (human researchers using the tools)
- SFT on successful trajectories with completion-only masking (mask tool outputs)
- This teaches the agent tool-use syntax and basic research workflow

2. Phase 2 — GRPO Training (3000 steps):

- Prompt pool: 500 research questions across 10 domains (ML, NLP, CV, systems, etc.)
- Per question: generate $N = 4$ complete research trajectories
- Score each trajectory with multi-component reward
- GRPO advantage: $\hat{A}_i = (R_i - \mu_G)/\sigma_G$
- Update policy with clipped objective (clip $\epsilon = 0.2$, KL $\beta = 0.05$)
- Curriculum: start with simple “summarize findings on X” tasks, progress to “design and run experiment on X”

3. Phase 3 — Rejection Sampling Fine-Tuning (200 steps):

- Generate 16 trajectories per hard question, keep top-2 by reward
- SFT on these high-quality trajectories
- Stabilizes performance on the hardest research tasks

12.7.7 Example Trajectory: Full MDP Trace

To illustrate how all the MDP components work together in practice, we trace a complete research episode from question to submission, annotating each step with its formal MDP elements.

Research Agent Trajectory — Full MDP Trace

Research question: “Does increasing GRPO group size beyond $N = 16$ improve math reasoning performance?”

Below is a complete trajectory with explicit MDP notation for each step: state s_t , action a_t , transition T , observation o_t , and cumulative reward.

t	Element	Description
	a_0	think: “I need papers on GRPO group size ablations, then a controlled experiment.”
	o_0	Internal reasoning (no tool output). Shaping reward: $r_0 = 0$.
1	s_1	$s_0 \oplus (a_0, o_0)$ — context now includes reasoning plan.
	a_1	search_papers (query="GRPO group size ablation math")
	o_1	Returns 5 papers: [DAPO, DeepSeek-Math, Dr. GRPO, ...]. $r_1 = 0$.
2	s_2	$s_1 \oplus (a_1, o_1)$ — search results in context.
	a_2	read_paper (id="DAPO", sections=["experiments", "ablations"])
	o_2	Key finding: “diminishing returns above $N = 16$ on code tasks; math untested.” $r_2 = 0$.
3	s_3	$s_2 \oplus (a_2, o_2)$ — literature gap identified.
	a_3	think: “DAPO tested code only. I’ll test $N \in \{4, 8, 16, 32, 64\}$ on GSM8K with Llama-3 8B. Control: same total training tokens.”
	o_3	Hypothesis formed. $r_3 = 0$.
4	s_4	$s_3 \oplus (a_3, o_3)$ — experiment plan in context.
	a_4	write_code ("train_grpo_gsm8k.py") — 85-line script with argparse for group_size, proper seed, eval on GSM8K test split.
	o_4	File written successfully. $r_4 = 0$.
5	s_5	$s_4 \oplus (a_4, o_4)$ — code available in workspace.
	a_5	execute_code ("python train_grpo_gsm8k.py -N 4 -seed 42")
	o_5	Accuracy: 67.2% (CI: 65.1–69.3). $r_5 = +0.1$ (successful execution).
6–8	a_{6-8}	Execute with $N = 8, 16, 32 \rightarrow$ results: 71.4%, 74.8%, 75.9%. Each $r_t = +0.1$.
9	s_9	Context includes 4 data points.
	a_9	execute_code ("python train_grpo_gsm8k.py -N 64 -seed 42")
	o_9	Accuracy: 75.6% (CI: 73.4–77.8) — slight <i>decrease</i> from $N = 32$! $r_9 = +0.1$.
10	s_{10}	All 5 results available.
	a_{10}	compute_stats (test="paired_t_test", data=[67.2, 71.4, 74.8, 75.9, 75.6], groups=[4,8,16,32,64])
	o_{10}	$p < 0.01$ for $N = 4$ vs $N = 16$; $p = 0.72$ for $N = 32$ vs $N = 64$ (not significant). $r_{10} = 0$.
11	s_{11}	Statistical evidence for diminishing returns.
	a_{11}	plot_data (x=[4,8,16,32,64], y=[67.2,71.4,74.8,75.9,75.6], xlabel="Group Size N", ylabel="GSM8K Accuracy (%)", title="GRPO Group Size vs Math Reasoning")
	o_{11}	Plot saved to results/group_size_curve.png . $r_{11} = 0$.
12	s_{12}	Full evidence assembled.
	a_{12}	write_report: 4-section report (Introduction, Method, Results with figure, Conclusion: “ $N = 16$ –32 is optimal; $N = 64$ shows no gain and increases compute 4×”).
	o_{12}	Report written (2,100 words). $r_{12} = 0$.
13	a_{13}	submit — episode terminates.
	R_{terminal}	LLM-judge scores: quality=8/10, code correct, novel (extends DAPO to math), 13 steps, all sections present.

Terminal reward computation:

$$\begin{aligned}
 R &= \underbrace{0.30 \times \frac{8}{10}}_{\text{quality}} + \underbrace{0.30 \times 1.0}_{\text{correct}} + \underbrace{0.15 \times \frac{7}{10}}_{\text{novelty}} + \underbrace{0.15 \times (1 - \frac{13}{100})}_{\text{efficiency}} + \underbrace{0.10 \times 1.0}_{\text{format}} \\
 &= 0.24 + 0.30 + 0.105 + 0.13 + 0.10 = \mathbf{0.875}
 \end{aligned}$$

Intermediate shaping total: $5 \times (+0.1) = +0.5$ (5 successful code executions).

GRPO context: This trajectory scored highest among the $N = 4$ group (others scored 0.61, 0.72, 0.53). GRPO advantage:

$$\hat{A} = \frac{0.875 - \bar{R}}{\sigma_R} = \frac{0.875 - 0.684}{0.129} = +1.48 \quad (\text{strongly reinforced})$$

Key MDP properties illustrated:

- **Deterministic T :** Each tool call produces a predictable state extension ($s_{t+1} = s_t \oplus (a_t, o_t)$).
- **Sparse terminal reward:** The real quality signal comes only at **submit**; intermediate shaping is small.
- **Long horizon:** 13 steps with $\gamma = 1.0$ (no discounting for episodic tasks).
- **Self-correction opportunity:** At step 9, the agent observes $N = 64$ doesn’t improve — adjusts its conclusion accordingly rather than cherry-picking.
- **Action diversity:** Mix of reasoning (**think**), information gathering (**search**, **read**), execution (**write_code**, **execute**), analysis (**compute_stats**, **plot**), and output (**write_report**, **submit**).

12.7.8 Key Design Decisions and Tradeoffs

Table 12.8: Design decisions for the research agent, mapped to paper sections.

Decision	Paper Section	Rationale
QLoRA ($r = 32$)	LoRA section	72B model; full fine-tune too expensive. $r = 32$ for complex reasoning.
GRPO (not PPO)	GRPO section	No value model needed; research quality is hard to predict mid-trajectory.
Sparse terminal reward	Reward Shaping	Research quality only measurable at completion; intermediate shaping minimal.
$N = 4$ trajectories	GRPO Group Size	Balance: enough diversity for ranking, not too expensive (100-step trajectories).
128K context	Flash Attention	Long trajectories with paper contents + code + results.
vLLM + prefix caching	vLLM section	System prompt + research question shared across 4 rollouts.
Curriculum training	Agentic RL	Start simple (literature review) → hard (design + execute experiments).
LLM-as-judge reward	Reward Models	Research quality is subjective; LLM judge is more flexible than rule-based.

12.7.9 Evaluation

Research Agent Evaluation Framework

- **Held-out questions (50):** Research questions unseen during training, covering diverse domains.
- **Human evaluation:** Domain experts rate reports on a 1–5 scale (quality, correctness, actionability).
- **Reproducibility:** Re-run the agent’s code from the report; verify results match.
- **Comparison baselines:** (1) Zero-shot GPT-4 + tools (no RL training), (2) SFT-only

agent, (3) Human researchers.

- **Efficiency metric:** Steps-to-completion normalized by task difficulty.

Expected results (based on similar agentic RL work):

Agent	Report Quality (1–5)	Avg Steps
Zero-shot GPT-4 + tools	2.8	25
SFT-only	3.4	18
GRPO-trained (ours)	4.1	14
Human researcher	4.5	N/A

12.7.10 Lessons and Failure Modes

Common Failures in Research Agent Training

- **Infinite loops:** Agent repeatedly searches for papers without progressing. *Fix:* Step budget + penalty for repeated tool calls with same arguments.
- **Code debugging spirals:** Agent spends 20+ steps fixing a single bug. *Fix:* Cap retries at 3; if code fails 3 times, abandon approach and try alternative.
- **Hallucinated citations:** Agent invents paper titles/results. *Fix:* Verify all citations exist via tool output; penalize unverifiable claims.
- **Premature submission:** Agent submits incomplete reports to avoid penalty for long trajectories. *Fix:* Minimum quality threshold ($R > 0.4$) to count as valid submission; below threshold is treated as failure.
- **Reward hacking the judge:** Agent learns to produce text that scores high with the LLM judge but is scientifically shallow. *Fix:* Rotate judge models; include human eval in the reward periodically.

12.8 State-of-the-Art RL for LLM Agents

RL techniques for LLM agents focus on **on-policy policy gradients** combined with **fine-grained credit assignment**. Because agents execute complex multi-turn trajectories involving tool interactions, API queries, and code execution, standard single-turn alignment algorithms must be heavily modified.

12.8.1 Dominant Baseline: GRPO for Agents

Popularized by DeepSeek-R1 [72], **GRPO** [323] is rapidly becoming the standard for agentic training. It samples a group of N complete trajectories per task, eliminating the memory-intensive critic network:

For a task prompt q , GRPO samples N agentic trajectories $\{o_1, o_2, \dots, o_N\}$ from $\pi_{\theta_{\text{old}}}$. The advantage for each trajectory is computed by normalizing its reward relative to the group:

$$A_i = \frac{r(o_i) - \frac{1}{N} \sum_{j=1}^N r(o_j)}{\text{std}(r(o_1), \dots, r(o_N))} \quad (12.18)$$

The GRPO objective with KL regularization:

$$L_{\text{GRPO}}(\theta) = \frac{1}{N} \sum_{i=1}^N \min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}, 1-\epsilon, 1+\epsilon \right) A_i \right) - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \quad (12.19)$$

Why GRPO Dominates Agentic Training

- **No critic:** Saves 50% GPU memory — critical when agent trajectories already consume massive context windows (32K–128K tokens).
- **Natural fit:** Agent tasks often have binary verifiable rewards (tests pass/fail, goal achieved/not) — perfect for group-relative normalization.
- **Exploration:** Sampling N diverse trajectories per task naturally explores different tool-use strategies.

12.8.2 PPO for Interactive Agents

PPO [319] remains valuable for agents operating in highly stochastic environments where step-level value estimation helps. The critic provides per-step advantage signals, enabling finer credit assignment when tool outputs are unpredictable:

- Step-level advantage estimation via GAE handles variable-length tool outputs
- Value head learns to predict “how likely is this trajectory to succeed from here”
- More stable when external tools return catastrophic errors that spike reward variance
- Trade-off: requires $2\times$ memory (critic) but provides denser learning signals

12.8.3 Fine-Grained Turn-Level Credit Assignment

The core challenge in agentic RL is the **sparse reward problem**. If an agent executes 20 tool actions and finally fails a unit test, a terminal reward of 0 punishes all 20 actions equally. Modern solutions:

Reinforcement Learning from Verifiable Rewards (RLVR)

Reward the model at **deterministic intermediate checkpoints**:

- Bash command compiles successfully $\rightarrow +0.1$
- Browser agent targets correct HTML element $\rightarrow +0.2$
- SQL query returns non-empty results $\rightarrow +0.1$
- Final test suite passes $\rightarrow +1.0$ (terminal)

Intermediate rewards provide gradient signal to *every* step, not just the final one. This dramatically accelerates learning by $3\text{--}5\times$ compared to sparse-only rewards.

Multi-Turn Trajectory Slicing

Frameworks split a multi-turn agent run into individual, independent steps. A credit assignment module isolates the **exact sub-step that broke the trajectory**:

1. Replay the successful prefix (steps $1\text{--}k$)
2. Identify the first divergence point (step $k + 1$ where it went wrong)
3. Assign negative reward only to that specific step
4. Assign neutral/positive rewards to correct prefix steps

This enables surgical policy updates without degrading already-correct behavior.

12.8.4 Alternative Paradigms

- **Iterative STaR (Self-Taught Reasoner)** [418]: Rather than continuous RL, use iterative offline loops. Generate trajectories → filter failures → SFT on successes → repeat. Simple to scale, avoids RL instability. Each iteration bootstraps reasoning ability.
- **Reinforcement World Model Learning (RWML)** [414]: To combat reward hacking, train agents to predict the *semantic consequence* of their actions. The agent receives an auxiliary reward for accurately predicting how environment state will change (e.g., predicting database table changes before executing SQL). This forces genuine understanding over superficial reward-gaming.
- **LATS (Language Agent Tree Search)** [438]: Apply Monte Carlo Tree Search over agent action sequences. At each step, expand multiple candidate actions, simulate their outcomes, and backpropagate rewards through the tree. Combines RL value estimation with search-time compute scaling.

12.8.5 Core Methodology Comparison

Table 12.9: Comparison of RL paradigms for LLM agents.

Method	Reward Density	Memory Cost	Primary Advantage
GRPO [323]	Sequence / final metric	Low (no critic)	Massive GPU memory reduction; simple implementation
PPO [319]	Step-by-step (GAE)	High (critic needed)	Fine-grained credit assignment; stable in noisy envs
Iterative STaR [418]	Sparse (filtered binary)	Minimal (SFT only)	Simple to scale; avoids RL optimization instability
RWML [414]	Dense (predictive)	Medium	Mitigates reward hacking via world modeling
LATS [438]	Backpropagated	High (tree expansion)	Best quality per task; scales with inference compute

12.9 Interactive RL Environments for Agent Training

The dominant paradigm in early agentic RL relied on **static datasets**: curated trajectories, offline preference pairs, or fixed benchmark suites. While tractable, this approach has a fundamental limitation—agents trained on static data cannot discover novel strategies through environment interaction, and reward signals are limited to what was anticipated at dataset creation time. The field has shifted decisively toward **interactive RL environments**: live simulators where agents execute actions, receive feedback, and update policies based on real outcomes.

Static vs. Interactive Training

- **Static (SFT/DPO)**: Agent learns from fixed trajectories. No exploration. Reward is implicit in data curation. Ceiling: human demonstrator quality.
- **Interactive RL**: Agent explores environment, receives verifiable rewards. Can discover strategies beyond the training distribution. Ceiling: environment reward quality.

The shift mirrors the transition from imitation learning to RL in robotics—interactive environments unlock qualitatively new capabilities.

12.9.1 NeMo Gym (NVIDIA)

NeMo Gym [271] is NVIDIA’s framework for interactive RL environments designed specifically for LLM agent training. Its architecture cleanly separates the **agent** (the LLM policy) from the

environment (the task simulator), enabling independent scaling of each component.

Key design principles:

- **Multi-turn rollouts:** Environments support extended interaction sequences, not just single-step queries. The agent can call tools, receive results, and continue reasoning across many turns.
- **Tool-calling verification:** Rewards are computed by executing the agent’s tool calls and verifying outcomes against ground truth—providing dense, reliable signal.
- **Decoupled agent/environment:** The environment runs as a separate service, allowing heterogeneous compute allocation (e.g., GPU for the agent, CPU for environment simulation).

Nemotron 3 Super was trained using NeMo Gym with 21 environment configurations spanning math, code, tool use, and multi-turn dialogue, generating 1.2M rollouts. The diversity of environments is critical: agents trained on a single environment type tend to overfit its reward structure.

12.9.2 RLFactory

RLFactory [369] is a plug-and-play framework for multi-round tool-use RL, designed to minimize the engineering overhead of connecting LLM policies to diverse tool environments.

RLFactory: Qwen3-4B Surpasses Larger Models

Using RLFactory with asynchronous tool calling and diverse reward signals:

- **Qwen3-4B** trained with RLFactory surpassed significantly larger models on Natural Questions.
- **6.8× throughput** improvement over synchronous tool-calling baselines via async execution.
- Diverse rewards (correctness, format, tool efficiency) prevent reward hacking on any single signal.

The key insight: small models with high-quality interactive training can outperform large models trained on static data, because interactive RL allows the model to discover tool-use strategies that no human demonstrator would think to include in a dataset.

12.9.3 MOSAIC: Safety in Agentic Training

As agentic RL environments become more capable, safety becomes a first-class concern. **MOSAIC** [50] (March 2026) addresses the problem of training agents that are both capable and safe in interactive environments.

The core MOSAIC framework introduces a **plan** → **check** → **act or refuse** loop:

1. **Plan:** The agent generates a candidate action sequence.
2. **Check:** A safety verifier evaluates the plan against a constraint set (harm categories, policy rules).
3. **Act or Refuse:** If the plan passes, execute; otherwise, generate a refusal with explanation.

Safety is trained via **trajectory-level preference learning**: rather than labeling individual actions as safe/unsafe, MOSAIC labels entire trajectories, capturing the cumulative safety impact of multi-step plans. This is critical because individual actions that appear benign can compose into harmful sequences.